C++ 对象内存模型实验报告

2025 年 3 月 16 日

1 实验内容

本实验通过设计包含多种类型成员的类，观察不同作用域对象的存储特性，验证以下内容：

1、定义一个类，其中有静态数据成员、各种类型非静态数据成员（含字符指针），甚至包括引用（可选），静态和非静态成员函数（含分配空间的构造函数、析构函数）。

2、定义全局对象、main 函数中局部对象、另一个被 main 调用的外部函数 func 中定义局部对象（可以是形参）、main 函数中动态创建对象，每种对象至少 2 个。观察、分析各种对象地址。

3、输出对象中各个静态与非静态数据成员的值、地址、对象的存储空间大小等信息。由此理解对象的本质、静态数据成员是本类对象共享一份拷贝等问题。此外，应观察对齐现象。

4、（可选）输出对象的每个字节，以揭示引用的实现方法。

5、对于上述各种对象，输出静态、非静态成员函数地址，以及 main、func 等外部函数的地址，并分析。

2 代码结构

核心类定义：

```
#include <iostream>
#include <cstring>
#include <iomanip>
using namespace std;
class A {
    static int static_mem; //静态成员
    int normal_mem;        //非静态成员
    char* str_ptr;         //字符指针
    int reference;         //引用指向值
    int& reference_mem;    //引用
public:
    A(int b, const char* c, int d)
     : normal_mem(b), reference(d), reference_mem(reference) {
        //构造函数
        str_ptr = new char[strlen(c) + 1];
        strcpy(str_ptr, c);
        static_mem++; //测试静态成员
    };
    ~A() {
        //析构函数
        delete[] str_ptr;
        static_mem--; //测试静态成员
```

```
    }
    static void static_func() {} //静态成员函数
    void normal_func() {}        //普通成员函数
    void print_memory() const {
        cout << setw(8) << "Object at: " << setw(10) << this << " Size: " << sizeof(*this
            ) << endl;                //打印当前对象地址与空间大小
        cout << setw(8) << "  static_mem: " << setw(15) << &static_mem << " (" <<
            static_mem << ")" << endl;       //打印静态成员
        cout << setw(8) << "  normal_mem: " << setw(15) << &normal_mem << " (" <<
            normal_mem << ")" << endl;       //打印非静态成员
        cout << setw(8) << "  str_ptr:    " << setw(15) << (void*)str_ptr << " -> " <<
            str_ptr << endl;           //打印字符指针
        cout << setw(8) << "  ref_mem:    " << setw(15) << &reference_mem << " -> " <<
            reference_mem << endl;      //打印引用成员
        size_t offset_normal = reinterpret_cast<size_t>(&normal_mem) - reinterpret_cast<
            size_t>(this);
        size_t offset_str = reinterpret_cast<size_t>(&str_ptr) - reinterpret_cast<size_t
            >(this);
        size_t offset_ref = reinterpret_cast<size_t>(&reference_mem) - reinterpret_cast<
            size_t>(this);
        cout << "  Member offsets: " << offset_normal << "/" << offset_str << "/" <<
            offset_ref << endl;           //研究对齐现象
    }
    static void print_static() {
        cout << setw(8) << "\n  Final static_mem: " << setw(15) << &static_mem << " (" <<
             static_mem << ")" << endl;       //用于打印最终的静态成员
    }
};

int A::static_mem = 0; //初始化静态成员
```

全局对象与外部函数：

```
    //全局对象
A global1(1, "global1", 2);
A global2(3, "global2", 4);

void func() {
    //外部函数局部对象
    A local1(10, "local1", 100);
    A local2(20, "local2", 200);
    cout << "\n[Function Local Objects]" << endl;
    local1.print_memory();
    local2.print_memory();
}
```

main 函数调用：

```
int main() {
    cout << "======== Memory Analysis ========\n";
    cout << "[Global Objects]" << endl;
    global1.print_memory();
    global2.print_memory();
```

```
6
7       // main函数局部对象
8       A local1(30, "main_local1", 300);
9       A local2(40, "main_local2", 400);
10
11      cout << "\n[Local Objects]" << endl;
12      local1.print_memory();
13      local2.print_memory();
14
15      // 动态对象
16      A* dynamic1 = new A(50, "dynamic1", 500);
17      A* dynamic2 = new A(60, "dynamic2", 600);
18
19      cout << "\n[Dynamic Objects]" << endl;
20      dynamic1->print_memory();
21      dynamic2->print_memory();
22
23      func(); // 引入外部函数
24
25      union SafeConverter {
26      void (A::*member_ptr)();
27      void* raw_ptr;
28  };
29      // 函数地址分析
30      cout << "\n======== Function Address ========\n";
31      SafeConverter sc;
32      sc.member_ptr = &A::normal_func;
33      cout << setw(20) << "  Member function:  " << setw(10) << sc.raw_ptr << endl;
34      cout << setw(20) << "  Static function:  " << setw(10) << (void*)&A::static_func <<
            endl;
35      cout << setw(20) << "  main function:    " << setw(10) << (void*)&main << endl;
36      cout << setw(20) << "  func:             " << setw(10) << (void*)&func << endl;
37
38      delete dynamic1;
39      delete dynamic2;
40
41      A::print_static();
42      return 0;
43  }
```

3 输出结果

```
1       ======== Memory Analysis ========
2       [Global Objects]
3       Object at: 0x7ff68e678060 Size: 32
4         static_mem:  0x7ff68e678040 (2)
5         normal_mem:  0x7ff68e678060 (1)
6         str_ptr:           0x6a7af0 -> global1
7         ref_mem:     0x7ff68e678070 -> 2
8         Member offsets: 0/8/16/16
```

```
    Object at: 0x7ff68e678080 Size: 32
      static_mem:  0x7ff68e678040 (2)
      normal_mem:  0x7ff68e678080 (3)
      str_ptr:           0x6a7b30 -> global2
      ref_mem:     0x7ff68e678090 -> 4
      Member offsets: 0/8/16/16

    [Local Objects]
    Object at:   0x5ffe80 Size: 32
      static_mem:  0x7ff68e678040 (4)
      normal_mem:        0x5ffe80 (30)
      str_ptr:           0x6b2f60 -> main_local1
      ref_mem:           0x5ffe90 -> 300
      Member offsets: 0/8/16/16
    Object at:   0x5ffe60 Size: 32
      static_mem:  0x7ff68e678040 (4)
      normal_mem:        0x5ffe60 (40)
      str_ptr:           0x6b2fa0 -> main_local2
      ref_mem:           0x5ffe70 -> 400
      Member offsets: 0/8/16/16

    [Dynamic Objects]
    Object at:   0x6b51f0 Size: 32
      static_mem:  0x7ff68e678040 (6)
      normal_mem:        0x6b51f0 (50)
      str_ptr:           0x6b5240 -> dynamic1
      ref_mem:           0x6b5200 -> 500
      Member offsets: 0/8/16/16
    Object at:   0x6ac9b0 Size: 32
      static_mem:  0x7ff68e678040 (6)
      normal_mem:        0x6ac9b0 (60)
      str_ptr:           0x6aca00 -> dynamic2
      ref_mem:           0x6ac9c0 -> 600
      Member offsets: 0/8/16/16

    [Function Local Objects]
    Object at:   0x5ffdf0 Size: 32
      static_mem:  0x7ff68e678040 (8)
      normal_mem:        0x5ffdf0 (10)
      str_ptr:           0x6add90 -> local1
      ref_mem:           0x5ffe00 -> 100
      Member offsets: 0/8/16/16
    Object at:   0x5ffdd0 Size: 32
      static_mem:  0x7ff68e678040 (8)
      normal_mem:        0x5ffdd0 (20)
      str_ptr:           0x6addd0 -> local2
      ref_mem:           0x5ffde0 -> 200
      Member offsets: 0/8/16/16

    ======== Function Address ========
      Member function:  0x7ff68e672de0
      Static function:  0x7ff68e672df0
```

```
61      main function:    0x7ff68e671507
62      func:             0x7ff68e671430
63
64      Final static_mem:  0x7ff68e678040 (4)
```

4 输出结果分析

1. 对象地址研究

对象按其作用域不同，在内存中的存储位置各异。全局对象存于数据段，地址较高且连续，像实验中的地址范围在 0x7ff68e678060 到 0x7ff618528080，间隔 32 字节，可能是因为内存对齐或编译器优化。局部对象位于栈区，地址呈现递减趋势，如 main 函数局部对象地址从 0x5ffe80 到 0x5ffe60，符合栈区向下增长的特性。动态对象则在堆区，地址较为分散，实验中为 0x6b51f0 和 0x6ac9b0，其内存分配由堆管理器负责。

2. 对象大小研究

对象大小为 32 字节，这是由其成员及内存对齐规则共同决定。对象包含 int normal_mem（4 字节）、char* str_ptr（指针变量 8 字节）、int reference（引用指向值）、int& reference_mem（实际为指针，8 字节）。由于要按最大成员的 8 字节进行内存对齐，便于读取数据，所以最终的实际布局为：normal_mem（4 字节）后填充 4 字节，接着是 str_ptr（8 字节），reference（4 字节）后填充 4 字节，reference_mem（8 字节），总计 $4+4+8+4+4+8=32$ 字节。

3. 静态成员分析

静态成员 static_mem 地址具有唯一性，所有对象的 static_mem 地址相同，都为 0x7ff68e678040，这体现了静态成员属于类且全局唯一的特性。其数值变化与对象构造和析构紧密相关，全局对象构造时，static_mem 从 0 变为 2（global1 和 global2 构造）；main 函数中局部对象构造使其从 2 变为 4（local1 和 local2）；动态对象构造后又从 4 变为 6（dynamic1 和 dynamic2）；func 函数中局部对象构造让其达到 8。随着动态对象和 func 函数中局部对象析构，static_mem 最终变为 4。

4. 非静态成员分析

非静态成员 normal_mem 的地址等于对象基地址,例如 global1 的 normal_mem 地址为 0x7ff68e678060，与对象地址一致，因为它是第一个非静态成员。不同对象的 normal_mem 值相互独立，不被其他成员所影响。如 global1.normal_mem = 1，global2.normal_mem = 3。引用指向值 reference 同理。

5. 字符指针分析

在代码中，输出字符指针地址时需强制转换为 void*，写成 (void*)str_ptr，为其指向的堆内存地址；若使用 &str_ptr，则会输出该变量本身地址（栈或对象内存）；而输出其指向内容时可直接使用 str_ptr，例如 cout << str_ptr 会显示字符串。像 global1.str_ptr 地址为 0x6a7af0，指向字符串 "global1"，动态对象的 str_ptr 地址则不同，如 0x6b5240 指向 "dynamic1"。

6. 引用成员分析

引用成员地址与被引用的成员地址相同（如 ref_mem 与 reference 均位于 this+16）。因引用是别名，不占用独立存储，直接绑定到目标变量，所以引用成员指向的值即为赋给 reference 的值，如 dynamic1 中引用成员：0x6b5200 -> 500。

7. 函数地址分析

成员函数地址与对象无关，所有对象共享同一成员函数实例，例如 A::normal_func 地址固定为 0x7ff68e672de0。静态函数地址可直接获取，如 &A::static_func。这些函数的地址也可以反映不同函数实现位置。

8. 对齐现象

成员按 8 字节对齐。引用成员不占独立空间，地址与目标变量相同。引用本质是别名，存储由编译器优化，输出显示其地址与目标一致。